

Programação de Computadores

Aula #15

Tutorial de Allegro

Como compilar? Como criar de telas?

Ciência da Computação - BCC e IBM

Prof. Vinícius Fülber Garcia

Sintaxe da Biblioteca

A Sintaxe da Biblioteca: Vamos agora entender os principais comandos do Allegro.

Veremos também a função `END_OF_MAIN()`, que é a função necessária para indicar o fim de main.

Para um estudo deste assunto iremos escrever um código totalmente novo.

Vamos carregar protótipos e as definições do Allegro, carregando o arquivo `allegro.h`:

```
#include <allegro.h>
```

Sintaxe da Biblioteca

- Como sabemos, ao declarar o arquivo de cabeçalho, teremos acesso às funções pré programadas que facilitam o desenvolvimento do código. O arquivo incluído foi `allegro.h` , que é a biblioteca gráfica a qual estamos estudando.
- Vamos agora declarar a função principal do nosso programa, a `main()`:

```
#include<allegro.h>  
int main() {}
```

Sintaxe da Biblioteca

- Agora vamos fazer as definições e chamadas de funções importantes para a utilização do Allegro. A primeira função que devemos chamar é `allegro_init()`, que inicializará as funções da biblioteca. Logo após esta função, devemos chamar uma função para interpretar os comandos dos dispositivos de entrada, no nosso caso, a função `install_keyboard()`, referente ao teclado, e se for necessário, outros dispositivos de entrada, como por exemplo o mouse. A função `install_keyboard()` interpreta os valores do teclado de forma diferente das funções de input padrão de outras bibliotecas, como por exemplo, `stdio.h` entre outras. Mais definições da função, serão abordadas no tópico referente a movimentação de bitmaps com o teclado. Vejamos agora a utilização das funções:

Sintaxe da Biblioteca

```
#include <allegro.h>
```

5

```
int main() {
```

```
    allegro_init();  
    install_keyboard();
```

```
}
```

Sintaxe da Biblioteca

- O próximo passo, está relacionado às definições de video. Vamos definir através da utilização da função `set_color_depth()`, qual será a profundidade de cores utilizada. Essa função pode receber valores pré estabelecidos de 8, 15, 16, 24 e 32 cores. Este número, é regido por RGB, ou seja, variação de tonalidade de 3 cores primárias que possibilitam a utilização de outras cores. O significado de RGB é na verdade Vermelho(Red), Verde(Green) e Azul(Blue), e é utilizado como padrão para preenchimento de cores em Bitmaps.

Sintaxe da Biblioteca

```
set_color_depth(32);
```

7

- A função `set_color_depth()` deve ser utilizada sempre antes da função `set_gfx_mode()`. Caso contrário, as cores de algum Bitmap, não serão exibidas corretamente.
- A próxima definição de vídeo a ser feita, é a definição da placa de vídeo e de sua resolução. Para isso utilizaremos a função `set_gfx_mode()`, que tem como parâmetro inicial a detecção do modo de vídeo, a definição da resolução em que a aplicação será exibida:

```
set_gfx_mode(GFX_AUTODETECT, 640 , 480 , 0 , 0 );
```

Sintaxe da Biblioteca

- Na linha acima, vemos a função sendo utilizada, com o parâmetro de detecção automática da placa de vídeo com uma resolução 640x480.

Os próximos campos, onde vemos dois zeros, são respectivamente a altura e largura virtuais da tela, ou seja, partes do mapa de um jogo que não aparecem no display.

Estas áreas virtuais são acessadas por meio de uma função de scowling, ou seja, rolamento de tela. Neste exemplo não utilizaremos a área virtual, por isso não daremos importância para ela agora.

Sintaxe da Biblioteca

- Veremos agora alguns dos modos de vídeo possíveis e suas descrições:

GFX_TEXT	retorna para o modo texto
GFX_AUTODETECT	faz com que o Allegro escolha o driver gráfico mais apropriado (normalmente tem problemas com placas de vídeo mais recentes)
GFX_VGA	modo VGA (320x200, 320x100, 160x120, 80x80)
GFX_MODEX	uma versão planar do modo VGA (tweaked)
GFX_VESA1	usa o driver VESA 1.x
GFX_VESA2B	usa o driver VBE 2.0 em modo banked
GFX_VESA2L	usa o driver VBE 2.0 com framebuffer linear
GFX_VESA3	usa o driver VBE 3.0
GFX_VBEAF	usa o acelerador de hardware API VBE/AF
GFX_XTENDED	usa o driver 640x480 unchained

Sintaxe da Biblioteca

- Como podemos perceber, os modos de video acima são definições divididas em duas partes, GFX, que se refere ao modo gráfico, e os modos propriamente ditos, como por exemplo AUTODETECT.

Porém, existe outro sub- parâmetro pode ser utilizado; caso você queira deixar seu jogo em modo fullscreen(tela cheia), você deve declarar o modo, e no final acrescentar o sub-parâmetro FULLSCREEN.

Caso não o faça, a função irá assumir default, executando seu jogos em uma janela. Veja exemplos:

```
set_gfx_mode(GFX_AUTODETECT, 640 , 480 , 0 , 0 );
```

```
set_gfx_mode(GFX_AUTODETECT_WINDOWED, 640 , 480 , 0 , 0 );
```

```
set_gfx_mode(GFX_AUTODETECT_FULLSCREEN, 640 , 480 , 0 , 0 );
```

Sintaxe da Biblioteca

- Uma vez definidos estes parâmetros, podemos iniciar uma rotina para inserir os gráficos.

Como estamos utilizando a biblioteca allegro.h, não podemos exibir texto da mesma forma que fazemos com cout ou printf(), pois não fazem parte desta biblioteca.

Para imprimir texto utilizaremos o seguinte comando:

```
textout(screen, font, "texto", 1,1, makecol(255,0,0 ) );
```

Sintaxe da Biblioteca

Exemplo:

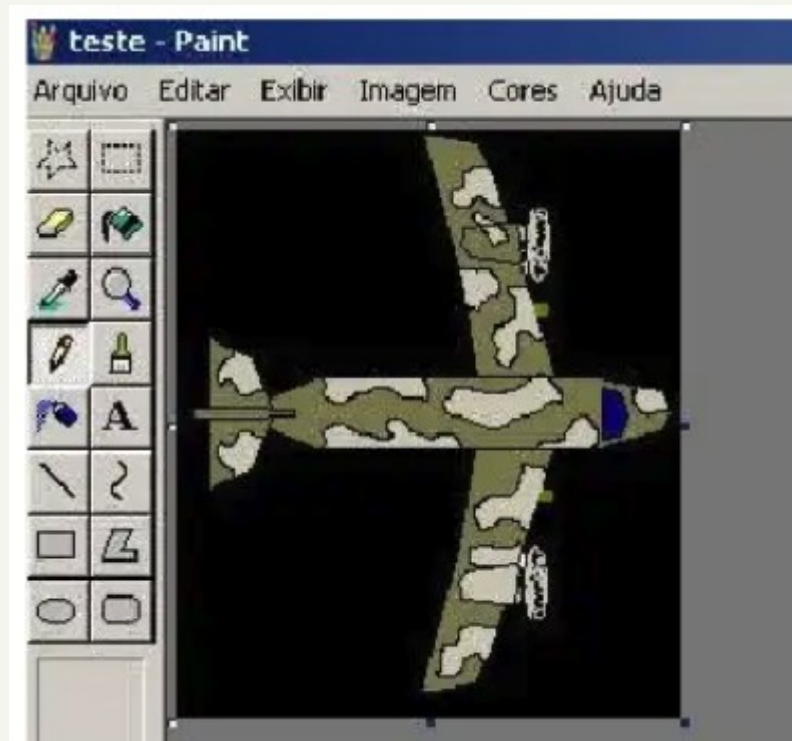
```
#include <allegro.h>
int main() {
    allegro_init();
    install_keyboard();
    set_color_depth(32);
    set_gfx_mode(GFX_SAFE, 800 , 600 , 0 , 0 );
    while (!key[KEY_ESC]) {
        textout(screen, font, "texto", 1,1, makecol(255,0,0 ) );
    };
    return 0;
}
```

Sintaxe da Biblioteca

- Agora vamos ver como fazer para inserir uma imagem na janela.

O procedimento é um pouco mais complexo, porém, é muito simples de se fazer.

O primeiro passo para se exibir uma imagem, é ter uma imagem, então, vamos criar uma para nosso exemplo:



Sintaxe da Biblioteca

O procedimento para adicionar a imagem são:

```
PALETTE pal; /* declaração uma variável tipo PALETTE */
```

```
BITMAP *img = load_bitmap("c:\\teste.bmp",pal);  
/* Ponteiro BITMAP carrega imagem */
```

```
draw_sprite(screen, img, 30,80 ); /* função para imprimir imagem */
```

Sintaxe da Biblioteca

1: Aqui temos a declaração da variável `pal` , tipo `PALETTE`, que é utilizada na linha seguinte também. Variáveis do tipo `PALETTE` , são utilizadas para definir as cores que o `BITMAP` está utilizando.

2: Nesta linha temos a declaração de um ponteiro do tipo `BITMAP` , que nomeamos `*img` . Esta variável recebe, através da função `load_bitmap()` , o arquivo de imagem que iremos carregar, `teste.bmp`, seguido do parâmetro `pal` , descrito acima

3: A função `draw_sprite()` é a função que carrega a imagem na tela. Seus parâmetros são similares às da função `textout()` , porém não utiliza os parâmetros `font` e `makecol(r,g,b)` . Mas em contrapartida, utiliza as declarações feitas nas duas linhas anteriores. Nesta linha, o parâmetro `img` é a imagem propriamente dita, e os números a seguir são coluna e linha respectivamente.

Sintaxe da Biblioteca

```
#include <allegro.h>
```

```
int main() {
```

```
    allegro_init();
```

```
    install_keyboard();
```

```
    set_color_depth(32);
```

```
    set_gfx_mode(GFX_AUTODETECT_WINDOWED,640,480,0,0);
```

```
    PALETTE pal;
```

```
    BITMAP *img = load_bitmap("c:\\teste.bmp",pal);
```

```
    while (!key[KEY_ESC]) {
```

```
        clear_bitmap(img);
```

```
        textout(screen, font, "Exemplo", 30,20, makecol(255,0,0 ) );
```

```
        draw_sprite(screen, img, 30,30 );
```

```
    };
```

```
    return 0;
```

```
}
```


Sintaxe da Biblioteca

- Um último comando deve ser incluso ao código para para que o programa execute corretamente.

A função `END_OF_MAIN()` faz com que o Allegro entenda que é o fim da função `main()`, ou seja, o fim do programa.

A não inclusão da função `END_OF_MAIN()` fará com que o DevC++ apresente uma mensagem de erro.

Sintaxe da Biblioteca

```
#include <allegro.h> /* Adiciona as funções do Allegro */
int main() {
    allegro_init(); /* função de inicialização do allegro */
    install_keyboard(); /* função de inicialização de teclado, agora dentro da main() */
    set_color_depth(32); /* função de profundidade de cores, definição em 32 cores */
    set_gfx_mode(GFX_AUTODETECT_WINDOWED,640,480,0,0); /* seleciona resolução */
    PALETTE pal; /* declaração uma variável tipo PALETTE */
    BITMAP *img = load_bitmap("c:\\teste.bmp",pal); /* Ponteiro BITMAP img */
    while (!key[KEY_ESC]) { /* Enquanto tecla diferente de ESC */
        clear_bitmap(img); /* Limpa o ponteiro */
        textout(screen, font, "Exemplo", 30,20, makecol(255,0,0 ) ); /* função para imprimir
texto */
        draw_sprite(screen, img, 30,30 ); /* função para imprimir imagem */
    }; /* fim do while() */
    return 0; /* Sai da função main() sem erros fazendo com que o programa finalize */
}
END_OF_MAIN() /* Finaliza a execução do Allegro neste programa */
```

Sintaxe da Biblioteca



Sintaxe da Biblioteca

- Desenhando Bitmaps: Agora vamos ver algumas das funções de desenho geométrico primitivo do Allegro.
- Estas funções são utilizadas para desenhar retas, círculos, retângulos e diversas outras formas através de pixels. O que se deve saber a respeito deste tipo de desenho, é a forma com que ele é exibido.
- Para desenhar uma imagem, é necessário criar um ponteiro do tipo BITMAP, que será utilizado como um buffer para a execução da função.
- Neste ponteiro, será definido o tamanho máximo da imagem que será desenhada.
- A definição é feita através da função `create_bitmap()`, que recebe como parâmetro dois números inteiros referente à resolução utilizada na janela. Iremos então criar um BITMAP:

Sintaxe da Biblioteca

```
#include <allegro.h>
```

```
int main() {  
    allegro_init();  
    install_keyboard();  
    set_color_depth(32);  
    set_gfx_mode(GFX_AUTODETECT_WINDOWED,640,480,0,0);  
    BITMAP *img = create_bitmap(640,480); /* Cria o BITMAP */  
    while (!key[KEY_ESC]) { };  
    return 0;  
}  
END_OF_MAIN();
```

Sintaxe da Biblioteca

- O próximo passo será definir a dimensão e a cor de nossa linha.

Para isto, iremos utilizar a função padrão de criação de linha do Allegro, `line()`, que recebe como parâmetro o ponteiro que criamos (`BITMAP`), os pontos de início e fim da linha (`int`), e a função de cor da linha (`makecol()`).

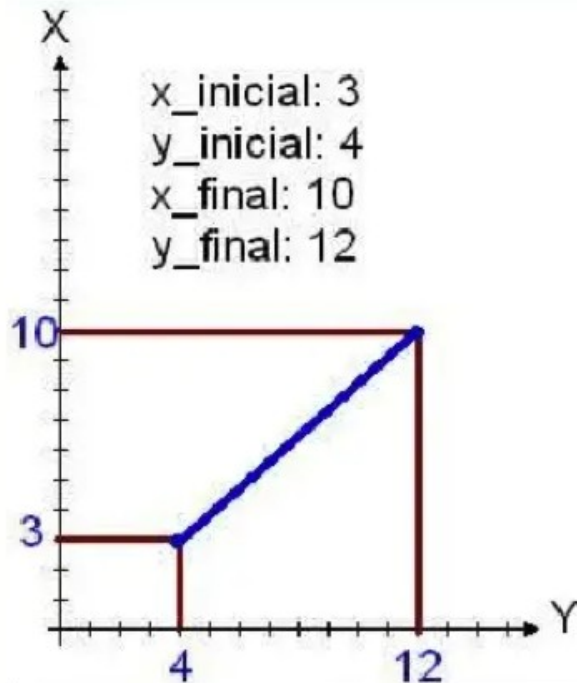
Desta forma a sintaxe da função seria esta:

```
line(BITMAP *img, int x_inicial, int y_inicial ,int x_final,int y_final, makecol(int,int,int ));
```

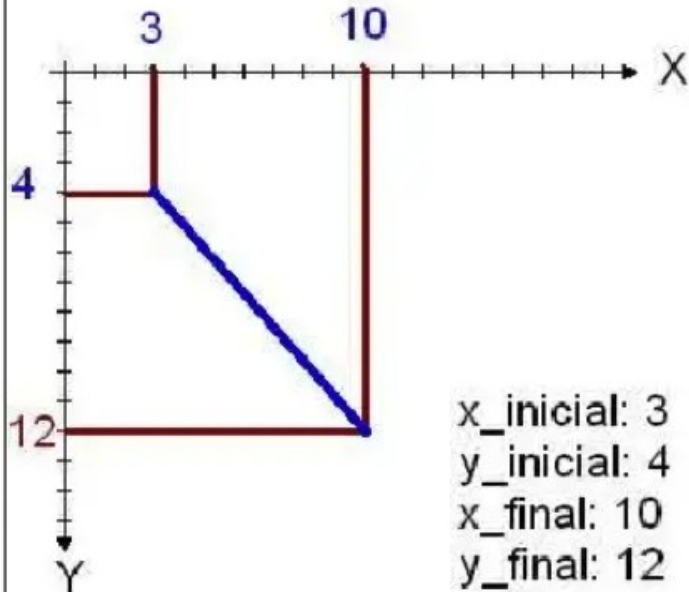
- E esta seria sua utilização:

```
line(img, 10,10, 100,100, makecol(255,0,0 ) )
```

Sintaxe da Biblioteca



CARTESIANO



BITMAP

Sintaxe da Biblioteca

```
#include <allegro.h>
```

```
int main() {
```

```
    allegro_init();
```

```
    install_keyboard();
```

```
    set_color_depth(32);
```

```
    set_gfx_mode(GFX_AUTODETECT_WINDOWED,640,480,0,0);
```

```
    BITMAP *img = create_bitmap(640,480); /* Cria o BITMAP */
```

```
    while (!key[KEY_ESC]) {
```

```
        clear_bitmap(img);
```

```
        line(img, 10,10, 100,100, makecol(255,0,0 ) ); /* Define dimensão e cor da linha */
```

```
    };
```

```
    Return 0;
```

```
}
```

```
END_OF_MAIN();
```


Sintaxe da Biblioteca

- Funções gráficas específicas para construção de figuras geométricas.
- Círculo: O círculo, é feito através da função `circle()`, que tem como parâmetro o ponteiro `BITMAP`, sua posição `X` e `Y`, seu raio e finalmente sua cor:

```
circle(img, 15,150,100, makecol(255,0,0 ) );
```

- Retângulo: O retângulo, é construído com a função `rect()`, e recebe como parâmetros um ponteiro de tipo `BITMAP`, as posições de `X` e `Y` do seu início que é no lado superior esquerdo, a posição de terminação, que é o lado inferior direito, e sua cor:

```
rect(img, 10,10, 100,100, makecol(255,0,0 ) );
```

Sintaxe da Biblioteca

- Preenchimento: Figuras como o retângulo e círculo, não tem preenchimento, sendo desenhadas apenas suas bordas, desta forma o programador terá que utilizar a função de preenchimento referente a figura para que a mesma tenha seu interior preenchido com alguma cor, caso contrário será apenas uma borda.

```
rectfill(img, 11,11,99,99, makecol(0,0,255 ) );
```

Exercício #16

1- Compile e rode os exercícios da aula.

2- Construa uma tela e dentro da tela desenhe um cubo.
Pesquise como fazer para mover o cubo pela tela.

• •

Obrigado!

Vinícius Fülber Garcia
inf.ufpr.br/vinicius/
viniciusfulber@ufpr.br

• •